

Universitatea Politehnica din București  
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

# PROIECT 3

## -Motor de căutare produse-

Studenti: Fawzy Rashwan Helal Nora-Mohamed 441G  
Nica Bianca-Elena 441G

Coordonator: Anamaria Dumitrescu

# Cuprins

|                                                  |    |
|--------------------------------------------------|----|
| 1. Introducere .....                             | 4  |
| 1.1 Scopul proiectului .....                     | 4  |
| 1.2 Prezentare generală a funcționalității ..... | 4  |
| 2. Tehnologii utilizate.....                     | 5  |
| 2.1 Python.....                                  | 5  |
| 2.2 Bibliotecile utilizate .....                 | 5  |
| 3. Setup și structura proiectului .....          | 7  |
| 4. Explicația detaliată a codului .....          | 8  |
| 4.1 Importarea bibliotecilor.....                | 8  |
| 4.2 Funcția `scrape_data` .....                  | 8  |
| 4.3 Funcția `sh_button` .....                    | 9  |
| 4.4 Tkinter GUI Setup .....                      | 10 |
| 5. Utilizare .....                               | 11 |
| 6. Concluzii .....                               | 14 |
| 7. Referințe.....                                | 15 |
| 8. Anexă.....                                    | 16 |

## Listă figuri

|                                                   |    |
|---------------------------------------------------|----|
| Figura 1 Interfață grafică Tkinter .....          | 11 |
| Figura 2 Căutare produs în fereastra Tkinter..... | 11 |
| Figura 3 Site-ul Carrefour căutare automată ..... | 12 |
| Figura 4 Scraping.....                            | 13 |
| Figura 5 Fișierul data.xlsx.....                  | 13 |

# 1. Introducere

Proiectul este la bază un instrument de web scraping, oferind utilizatorilor posibilitatea de a extrage și compila informații despre produse. Scopul central al codului furnizat constă în facilitarea procesului de extragere a datelor de pe site-ul Carrefour, cu accent special pe identificarea și colectarea informațiilor despre produse în funcție de termenii de căutare furnizați de utilizator. La finalul acestui proces, informațiile extrase sunt salvate într-un fișier Excel, oferind astfel o soluție practică pentru utilizatori.

## 1.1 Scopul proiectului

Codul funcționează ca un web scraper, permițând utilizatorilor să extragă cu ușurință numele produselor de pe site-ul Carrefour. Scriptul eficientizează colectarea de informații relevante prin automatizarea procesului de căutare și de extragere a datelor, oferindu-le utilizatorilor posibilitatea de a obține rapid și fără dificultăți informațiile dorite despre produsele disponibile pe Carrefour.

## 1.2 Prezentare generală a funcționalității

Scriptul utilizează caracteristicile bibliotecii Selenium pentru web scraping [1], time pentru implementarea întârzierilor, pandas pentru manipularea eficientă a datelor, tkinter pentru construirea unei interfețe grafice (GUI) și Pillow (PIL) pentru manipularea imaginilor. Aceste biblioteci se asociază pentru a oferi o soluție cuprinzătoare și robustă pentru obținerea datelor despre produse de pe site-ul Carrefour.

Proiectul este organizat într-un mod care permite o funcționare eficientă și coerentă. Fișierul principal de script gestionează procesul de web scraping, în timp ce interfața grafică Tkinter [2] [3] furnizează o modalitate interactivă și prietenoasă pentru utilizatori de a iniția căutarea.

În continuare, se vor prezenta detaliat atât tehnologiile folosite, dependențele necesare, structura proiectului, funcțiile utilizate, cât și explicații ample privind modul de funcționare al codului.

## 2. Tehnologii utilizate

În această secțiune, vom explora în detaliu tehnologiile utilizate în proiect, cuprinzând atât limbajul de programare Python, cât și bibliotecile specifice precum Selenium, pandas, tkinter, Pillow (PIL), și time.

### 2.1 Python

Python este un limbaj de programare de înalt nivel și interpretat, apreciat pentru simplitatea și versatilitatea sa. Sintaxa sa clară și ușor de înțeles îl face accesibil pentru toate nivelurile de experiență. Python este utilizat pe scară largă în diverse domenii precum dezvoltarea web, analiza datelor, inteligența artificială și crearea de aplicații desktop.

### 2.2 Bibliotecile utilizate

- **Selenium**

Selenium [3] este o bibliotecă open-source pentru automatizarea interacțiunilor cu browserul web. Aceasta oferă o interfață simplă pentru controlul unui browser web și efectuarea de acțiuni automate, ceea ce o face esențială în dezvoltarea de teste automate pentru aplicații web sau pentru web scraping. Astfel, este o componentă esențială a proiectului de față.

În cadrul acestui proiect, Selenium este utilizat pentru a inițializa un WebDriver Chrome, furnizând o interfață între codul Python și browser. Această interfață permite scriptului să efectueze acțiuni precum navigarea pe site-ul Carrefour și interacțiunea cu diverse elemente ale paginii.

- **time**

Modulul "time" este utilizat pentru a introduce perioade de întârziere în scriptul de web scraping. Întârzierile sunt folosite strategic pentru a gestiona sincronizarea între acțiunile automate și încărcarea elementelor pe pagină, garantând că extragerea datelor se face în momente corespunzătoare, evitând astfel capturarea informațiilor incomplete sau eronate.

- **pandas**

Biblioteca Pandas [4] este utilizată pentru manipularea și stocarea datelor obținute prin web scraping cu Selenium. Aceasta va fi folosită pentru a organiza numele produselor extrase într-un format structurat (DataFrame) și pentru a salva datele într-un fișier Excel ("data.xlsx").

- **tkinter**

Biblioteca Tkinter [2] este folosită pentru a crea o interfață grafică (GUI) user-friendly. Această interfață permite utilizatorului să introducă un termen de căutare pentru produsele de pe site-ul Carrefour. Mai apoi, acționând un buton, se inițiază procesul de web scraping folosind Selenium și Pandas.

- **Pillow (PIL)**

Biblioteca Pillow (PIL) [5] este utilizată pentru procesarea imaginilor, cu accent pe încărcarea și redimensionarea imaginilor de fundal. Scopul acestei funcționalități este să îmbunătățească aspectul vizual al interfeței grafice create cu Tkinter.

### 3. Setup și structura proiectului

Proiectul necesită o configurare adecvată a mediului de lucru și instalarea dependențelor esențiale.

- Configurarea Mediului de Lucru: Python, Editor de Text sau IDE (PyCharm)
- Instalarea dependențelor proiectului (pot fi adăugate și direct din Python Interpreter) folosind:  
**pip install selenium pandas pillow**
- Descărcarea driver-ului browser-ului: Proiectul folosește Selenium pentru a automatiza browser-ul Chrome, [ChromeDriver](#) trebuie instalat. [6]
- Instalarea Bibliotecii Tkinter (dacă nu este deja instalată):  
**pip install tk**

#### Structura directorului proiectului

- venv/: Directorul conținând mediul virtual Python pentru izolarea dependențelor.
- script-ul proiectnou.py
  - Inițiază o instanță a browserului Chrome folosind Selenium WebDriver.
  - Navighează către site-ul Carrefour.
  - Interacționează cu elementele HTML pentru a introduce un termen de căutare și a iniția căutarea.
  - Parcurge paginile de rezultate și extrage numele produselor.
  - Salvează datele într-un fișier Excel folosind Pandas.
  - Crează o fereastră Tkinter cu un câmp de introducere pentru termenul de căutare și un buton de căutare.
- data.xlsx: Fișierul Excel în care se salvează datele extrase.
- imagini utilizate în proiect.

## 4. Explicația detaliată a codului

### 4.1 Importarea bibliotecilor

Codul începe prin importul bibliotecilor necesare, inclusiv Selenium pentru web scraping (modulul webdriver de la Selenium este utilizat pentru a inițializa un browser web pentru interacțiuni automate) [3] [6] [7], time pentru adăugarea de întârzieri, pandas pentru manipularea datelor, tkinter pentru crearea interfeței grafice și Pillow (PIL) [5] pentru lucrul cu imagini.

### 4.2 Funcția ``scrape_data``

#### 4.2.1 Rol

Funcția `scrape_data` [7] [8] servește drept nucleu al procesului de scraping a datelor web. Scopul său principal este de a extrage numele produselor de pe site-ul Carrefour pe baza unui termen de căutare furnizat de utilizator. Această funcție înglobează întregul flux de scraping, de la inițializarea browserului și navigarea pe site-ul web până la salvarea datelor obținute într-un fișier Excel [9].

#### 4.2.2 Parametri de Intrare

- `search_term (str)`:
  - Termenul de căutare specificat de utilizator pentru produsul dorit pe site-ul Carrefour.

#### 4.2.3 Funcționalitate generală

##### 1. Inițializarea browserului:

- Funcția începe prin a inițializa un Chrome WebDriver folosind Selenium, responsabil pentru automatizarea interacțiunilor web.

##### 2. Navigarea pe site-ul Carrefour [6]:

- WebDriver-ul navighează pe site-ul Carrefour (<https://carrefour.ro/>) pentru a începe procesul de scraping.

##### 3. Procesul de căutare:

- Funcția localizează câmpul și butonul de căutare de pe site folosind ID-urile și XPATH-urile HTML corespunzătoare.
- Introduce termenul de căutare furnizat `search_term` în câmpul de căutare și inițiază căutarea făcând click pe butonul de căutare.

##### 4. Extragerea datelor:

- Scriptul iterează prin până la 10 pagini de rezultate de căutare, extrăgând denumirile produselor de pe fiecare pagină.



- Denumirile produselor sunt extrase localizând elementele de produs de pe pagina curentă folosind XPATH-urile lor.

## 5. Gestionarea paginării:

- Scriptul folosește un indicator (``next_button_available``) pentru a urmări disponibilitatea butonului "Next" pentru paginare.
- Un bloc ``try-except`` gestionează situațiile în care butonul "Next" nu este găsit, asigurând o încheiere fără probleme când paginarea este completă.

## 6. Stocarea datelor:

- Denumirile produselor extrase sunt salvate într-o listă (`products`).
- Datele sunt structurate într-un DataFrame pandas (`data`) cu o coloană denumită "Products".
- DataFrame-ul este salvat într-un fișier Excel cu numele "data.xlsx".

## 7. Actualizarea GUI-ului Tkinter:

- Funcția actualizează un label în GUI-ul Tkinter pentru a indica că datele au fost extrase și salvate.

## 8. Închiderea Browserului:

- După finalizarea procesului de scraping, browserul Chrome este închis.

## 9. Închiderea GUI-ului Tkinter:

- GUI-ul Tkinter este închis pentru a încheia întregul proces.

### 4.3 Funcția ``sh_button``

#### 4.3.1 Rol

Funcția `sh_button` acționează ca funcție de apelare pentru butonul "Search" al interfeței grafice Tkinter [10]. Scopul său este de a prelua termenul de căutare introdus de utilizator, declanșând inițierea procesului de răzuire a datelor prin intermediul funcției `scrape_data`.

#### 4.3.2 Funcționalitate generală

##### 1. Recuperarea ``search_term``:

- Funcția recuperează termenul de căutare introdus de utilizator din widgetul de introducere a cuvintelor cheie din interfața grafică Tkinter.

##### 2. Inițierea extragerii de date:

- Se apelează funcția `scrape_data`, trecând ca argument termenul de căutare recuperat.

## 4.4 Tkinter GUI Setup [10]

### 4.4.1 Rol

Această secțiune inițializează fereastra Tkinter pentru interacțiunea cu utilizatorul. Definește titlul ferestrei, dimensiunile acesteia și creează diferite widget-uri pentru a facilita introducerea datelor și interacțiunea cu utilizatorul.

### 4.4.2 Funcționalitate generală

#### 1. Inițializarea Ferestrei Tkinter

- Creează o fereastră Tkinter (`root`) cu un titlu ales și dimensiuni de 840x490 de pixeli.

#### 2. Funcția de apel pentru butonul 'Search': `sh_button`

- Funcția `sh_button` servește ca funcție de apel pentru butonul 'Search' al GUI-ului Tkinter, explicată anterior.

#### 3. Încărcarea și redimensionarea imaginii de fundal

- Încarcă o imagine de fundal folosind biblioteca Pillow și o redimensionează pentru a se potrivi ferestrei Tkinter.

#### 4. Crearea de widget-uri Tkinter

- Creează diverse widget-uri Tkinter, cum ar fi etichete, câmpuri de introducere, butoane etc., pentru interacțiunea cu utilizatorul.
- Creează o etichetă pentru imaginea de fundal și o plasează în fereastra Tkinter, acoperind întreaga fereastră.

#### 5. Fundalul etichetei, plasarea acesteia și alte widget-uri

- Creează un cadru pentru alte widget-uri și îl plasează în centrul ferestrei Tkinter.
- Creează etichete, câmpuri de introducere și butoane pentru a permite utilizatorului să introducă date și să interacționeze cu aplicația.

#### 6. Bucla principală

- `root.mainloop()` inițiază bucla principală de evenimente pentru aplicația Tkinter, permițând GUI-ului să răspundă la interacțiunile utilizatorului

## 5. Utilizare

Scriptul de web scraping pentru Carrefour realizat are ca scop extragerea denumirilor produselor pe baza unui termen de căutare introdus de utilizator. Procesul implică interacțiunea cu interfața grafică realizată în Tkinter, navigarea automată pe site-ul Carrefour folosind Selenium, și extragerea datelor relevante. În continuare, se va ilustra functionalitatea proiectului.

### 1. Interfața grafică Tkinter:



Figura 1 Interfață grafică Tkinter

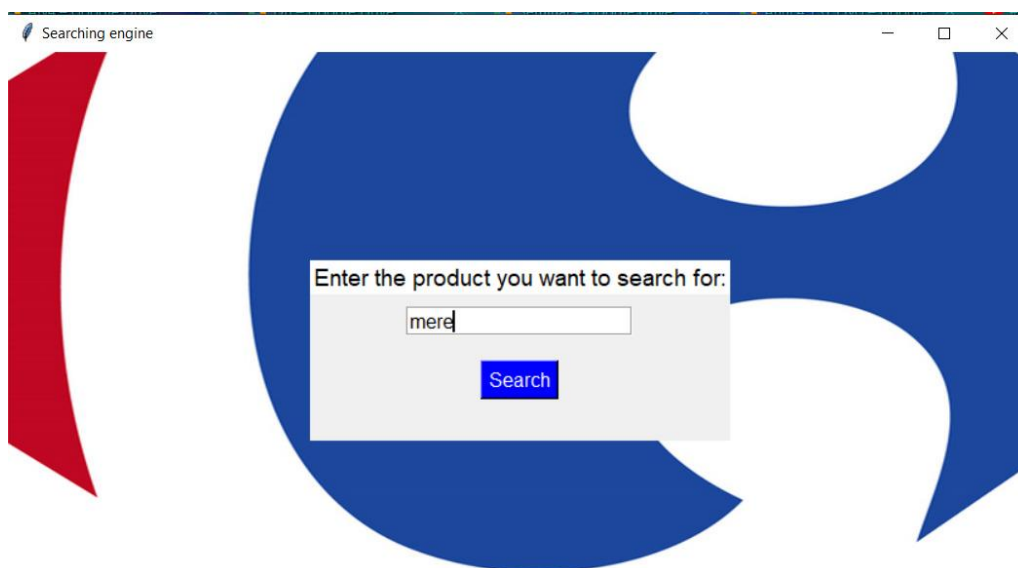


Figura 2 Căutare produs în fereastra Tkinter

- Utilizatorul introduce un termen de căutare în fereastra Tkinter.
- Apasă butonul "Search" pentru a iniția căutarea.

## 2. Navigare automată:

- Se deschide un browser Chrome automat și navighează către site-ul Carrefour.
- Site-ul web este manipulat automat pentru a introduce termenul de căutare și a iniția căutarea.

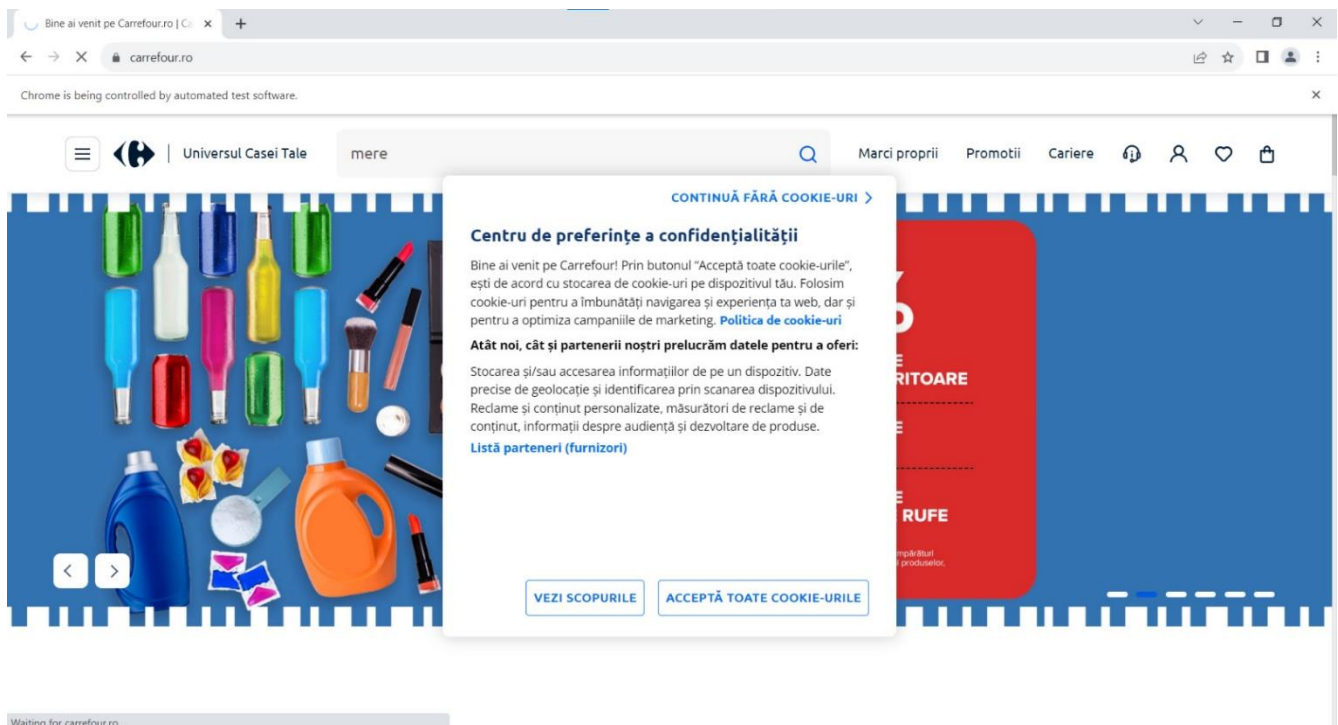


Figura 3 Site-ul Carrefour căutare automată

## 3. Web Scraping

- Scriptul utilizează Selenium pentru a itera prin paginile de rezultate și a extrage denumirile produselor.
- Gestionarea paginării este automată, asigurându-se că toate rezultatele sunt acoperite.

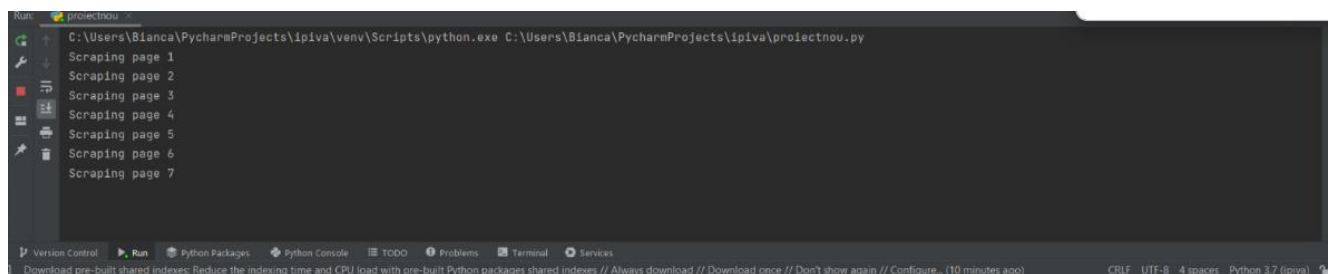


Figura 4 Scraping

#### 4. Salvare în Excel

- Denumirile produselor sunt stocate într-un fișier Excel (“data.xlsx”).
- Interfața Tkinter este actualizată pentru a indica finalizarea cu succes a procesului.

|    | A                                                                                  | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W |
|----|------------------------------------------------------------------------------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1  | Products                                                                           |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 2  | Gustare Organix de mere, capsuni si quinoa, 100g                                   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 3  | Spumant fara alcool Kidibul, mere Bio 0.75L                                        |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 4  | Suc Hipp de mere cu banana si ananas, +4 luni, 200 ml                              |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 5  | Suc natural Tymbarck de visine si mere 1 L                                         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 6  | Prajitura casei Alka cu mere confiate, 60 g                                        |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 7  | Hipp Suc de mere, 200 ml                                                           |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 8  | Piure Organix Eco de mere, cartof dulce si ananas, +12 luni, 100g                  |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 9  | Cozonac Boromir Hohoho cu nuca, mere si stafide 500g                               |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 10 | Cereale Organix Bio fara gluten din orez, porumb, mere, vitamina B1, +6 luni, 120g |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 11 | Smoothie Hipp cu fructe rosii, mere si banane 120 ml                               |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 12 | Otet balsamic din cidru de mere Ana Are, 250 ml                                    |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 13 | Biscuiti cu mere Merba 225 g                                                       |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 14 | Otet de mere De Nigris cu miere 500 ml                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 15 | Mousse Tedi de piersici, mere, banane si morcovi 100 g                             |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 16 | Suc de mere bio Pfanner 1L                                                         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 17 | Rondele Organix ecologice din orez cu mere, +7 luni, 40g                           |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 18 | Suc Ana Are de mere si catina 0.2                                                  |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 19 | Piure bio de mere si morcovi Hipp 125g                                             |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 20 | Hipp Piure mere si afine, 125 g                                                    |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 21 | Piure ecologic Hero Baby Solo cu lăut, Mere si Banane, 120g                        |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 22 | Otet din mere Ana Are 500 ml                                                       |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 23 | Jeleuri Bear Talpita Jungle coacaze negre si mere 20g                              |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 24 | Spuma bord satin, mere verzi, Carplan 500ml                                        |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 25 | Biscuiti Organix cu gem de capsuni si mere Goodies, 64g                            |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| 26 | Gustare Organix de mere, capsuni si quinoa, 100g                                   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |

Figura 5 Fișierul data.xlsx

## 6. Concluzii

Proiectul de web scraping dezvoltat reprezintă o soluție eficientă pentru colectarea și analizarea datelor referitoare la produsele disponibile pe platformele e-commerce, în cazul nostru Carrefour, simplificând procesul de căutare pentru utilizatori. Motorul de căutare, împreună cu interfața grafică, demonstrează utilitatea și aplicabilitatea tehnologiilor de automatizare în contextul manipulării datelor online.

O caracteristică distinctivă a proiectului este capacitatea de a gestiona volumul semnificativ de informații disponibile pe site-ul Carrefour, extrăgând și organizând datele relevante într-un format structurat. Salvarea automată într-un fișier Excel consolidează datele într-o formă tabulară ușor accesibilă, facilitând analiza ulterioară și compararea eficientă a produselor.

Prin intermediul interfeței grafice intuitive dezvoltate în Tkinter și a automatizării procesului de web scraping cu Selenium, utilizatorii au acces la un motor de căutare de produse care simplifică și accelerează semnificativ procesul de găsire a informațiilor dorite. Cu toate acestea, este important să se sublinieze sensibilitatea motorului de căutare la modificările aduse în structura paginei web sursă, ceea ce impune necesitatea actualizărilor constante pentru a menține funcționalitatea optimă.

În vederea îmbunătățirilor viitoare, se pot explora posibilități de extindere a funcționalităților, inclusiv integrarea unor mecanisme de filtrare avansată sau adăugarea unor opțiuni de sortare pentru rezultatele căutării. De asemenea, optimizările interfeței utilizatorului și integrarea unor elemente de experiență a utilizatorului ar putea consolida eficacitatea și atractivitatea motorului de căutare.

În concluzie, astfel de proiecte reprezintă un pas în direcția unei experiențe de cumpărături online mai fluentă și informată. Integrarea tehnologiilor de web scraping în acest context nu doar eficientizează procesul de selecție a produselor, ci și oferă utilizatorilor instrumente puternice pentru analiza detaliată și compararea atentă a ofertelor.

## 7. Referințe

- [1] "Web Scraping with Selenium and Python Tutorial + Example Project," Scrapfly, 10 01 2022. [Online]. Disponibil la: <https://scrapfly.io/blog/web-scraping-with-selenium-and-python/>. (Accesat la 18.10.2022)
- [2] I. t. Tkinter, "GeeksforGeeks," [Online]. Disponibil la: <https://www.geeksforgeeks.org/introduction-to-tkinter/>. (Accesat la 24.10.2023)
- [3] ScrapeOps, "The Python Selenium Guide - Web Scraping With Selenium," [Online]. Disponibil la: <https://scrapeops.io/selenium-web-scraping-playbook/python-selenium/>. (Accesat la 26.10.2023)
- [4] W3schools, "Pandas Tutorial," [Online]. Disponibil la: <https://www.w3schools.com/python/pandas/default.asp>. (Accesat la 24.10.2023)
- [5] "Image Processing With the Python Pillow Library," [Online]. Disponibil la: <https://realpython.com/image-processing-with-the-python-pillow-library/>. (Accesat la 24.10.2023)
- [6] "Using Chromedriver for Webscraping: A Comprehensive Guide," [Online]. Disponibil la: <https://webscrapeai.com/blog/using-chromedriver-for-webscraping-a-comprehensive-guide>. (Accesat la 25.10.2023)
- [7] "https://www.hackersrealm.net/post/scraping-products-from-amazon-selenium-python," [Online]. Disponibil la: <https://www.hackersrealm.net/post/scraping-products-from-amazon-selenium-python>. (Accesat la 03.11.2023)
- [8] "Scraping Products from Amazon using Selenium | Dynamic Website | Web Scraping Tutorial | Python," [Online]. Disponibil la: <https://www.youtube.com/watch?v=FXVjDTimQAg>. (Accesat la 05.11.2023)
- [9] "Web Scraping | Amazon Products Web Scraping Using Selenium from, Python to Excel | Part - 14," [Online]. Disponibil la: <https://www.youtube.com/watch?v=W0DnDq5yRVs&t=589s>. (Accesat la 07.11.2023)
- [10] "Tkinter Beginner Course - Python GUI Development," [Online]. Disponibil la: <https://www.youtube.com/watch?v=ibf5cx221hk>. (Accesat la 12.11.2023)

## 8. Anexă

```
from selenium import webdriver
from time import sleep
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait
import pandas as pd
import tkinter as tk
from PIL import Image, ImageTk

# Scriptul începe prin importarea bibliotecilor necesare, inclusiv Selenium pentru
web scraping, time pentru
# adăugarea întârzierilor, pandas pentru manipularea datelor, tkinter pentru
crearea GUI-ului și Pillow (PIL)
# pentru lucru cu imagini.

def scrape_data(search_term):
    """
    Această funcție primește un termen de căutare ca intrare.
    Inițializează un WebDriver Chrome folosind Selenium și navighează către site-ul
    Carrefour.
    Introduce termenul de căutare în bara de căutare și apasă butonul de căutare.
    Scriptul apoi iterează prin maximum 10 pagini de rezultate de căutare,
    extrăgând numele produselor de pe fiecare pagină.
    Numele produselor extrase sunt stocate într-o listă numită products.
    Datele sunt salvate într-un fișier Excel numit "data.xlsx" folosind pandas.
    Funcția actualizează, de asemenea, o etichetă în GUI-ul Tkinter pentru a indica
    că datele au fost extrase și salvate.
    Browser-ul Chrome este închis după ce procesul de scraping este complet.
    :param search_term: Termenul de căutare pentru site-ul Carrefour.
    :return: None
    """

    # Deschide browser-ul
    browser = webdriver.Chrome()
    browser.get('https://carrefour.ro/') # Navighează browser-ul către site-ul
    Carrefour.
    browser.maximize_window()
    browser.maximize_window()

    # Introducerea termenului de căutare și efectuarea căutării
    input_search = browser.find_element(By.ID,
                                         'search') # Localizează câmpul de căutare
    pe site folosind ID-ul său HTML.
    search_button = browser.find_element(By.XPATH,
                                         "//img[@src='https://cdn-
static.carrefour.ro/unified/assets/images/dist"
                                         "/icons/search.svg']") # Localizează
    butonul de căutare folosind XPATH-ul său.
    input_search.send_keys(search_term) # Introduce termenul de căutare furnizat
    în câmpul de căutare.
    sleep(1)
    search_button.click() # Apasă butonul de căutare pentru a iniția căutarea.
```



```

products = [] # Creează o listă goală pentru a stoca numele produselor.
next_button_available = True # Flag pentru a urmări disponibilitatea butonului
"Next" pentru paginare.

for i in range(10): # Extrage date de pe maximum 10 pagini
    print('Scraping page', i + 1)
    product_elements = browser.find_elements(By.XPATH,
                                              "//*[li/div/div/div/div/a") #
Găsește toate elementele de produs de pe
    # pagina curentă folosind XPATH-ul lor.

    # Extrage datele produsului și le adaugă la listă
    for product in product_elements:
        products.append(product.text)

    try: # Bloc try-except pentru a gestiona situațiile în care butonul
"Următorul" nu este găsit (o singură pagină de rezultate)
        if next_button_available:
            # Încearcă să apeleze butonul "Următorul" dacă este disponibil.
Dacă nu, iese din buclă.
            wait = WebDriverWait(browser, 10)
            next_page = wait.until(EC.element_to_be_clickable(
                (By.XPATH,
                 "//*[@id='maincontent']/div[3]/div[1]/div[2]/div[3]/div[1]/div/ul/li[5]")))
            next_page.click()
            sleep(2)
        else:
            break # Ieșirea din buclă dacă butonul "Next" nu mai este
disponibil
    except:
        next_button_available = False # Setează flagul la False dacă butonul
"Next" nu este găsit

    # Salvează datele în Excel
    data = pd.DataFrame({"Products": products}) # După ce s-au extras date de pe
mai multe pagini,
    # scriptul creează un DataFrame pandas numit data cu o coloană numită
"Products."
    data.to_excel("data.xlsx", index=False) # Salvează DataFrame-ul într-un fișier
Excel numit "data.xlsx."

    # Închide browser-ul
    browser.quit()

    result_label.config(text="Data scraped and saved to data.xlsx") # Actualizează
o etichetă de rezultat în GUI-ul Tkinter
    # pentru a indica că datele au fost extrase și salvate în "data.xlsx."
    root.destroy() # Închide GUI-ul Tkinter

root = tk.Tk()
root.title("Searching engine") # Numele ferestrei GUI
root.geometry("840x490") # Dimensiunile ferestrei GUI Tkinter

def sh_button():
    """
    Funcție de apel înapoi pentru butonul 'Caută' din GUI-ul Tkinter.
    Preia termenul de căutare introdus în widget-ul keyword entry

```

```

și inițiază procesul de scraping de date folosind funcția scrape_data.

:return: None
"""
search_term = keyword_entry.get() # Preia termenul de căutare introdus în
# widget-ul keyword_entry
scrape_data(search_term) # Apelează funcția scrape_data() cu termenul de
căutare

# Încarcă și redimensionează imaginea de fundal
image = Image.open("C:/Users/Bianca/Desktop/bg2.jpg") # Încarcă o imagine
folosind metoda din biblioteca Pillow (PIL).
bg_width, bg_height = root.winfo_screenwidth(), root.winfo_screenheight()
image = image.resize((bg_width, bg_height), Image.LANCZOS)
background_image = ImageTk.PhotoImage(image)

# Creează o etichetă pentru imaginea de fundal și o plasează în cadru
background_label = tk.Label(root, image=background_image)
background_label.place(relwidth=1, relheight=1)

# Creează un cadru pentru alte widget-uri
frame = tk.Frame(root)
frame.place(relx=0.5, rely=0.5, anchor=tk.CENTER)

# Alte widget-uri
keyword_label = tk.Label(frame, text="Enter the product you want to search for:",
font=("Helvetica", 14), bg="white")
keyword_label.pack()

keyword_entry = tk.Entry(frame, font=("Helvetica", 12), bd=2, relief=tk.GROOVE)
keyword_entry.pack(pady=10)

sr_button = tk.Button(frame, text="Search", font=("Helvetica", 12), bg="blue",
fg="white", relief=tk.RAISED,
command=sh_button)
sr_button.pack(pady=10)

result_label = tk.Label(frame, text="", font=("Helvetica", 12))
result_label.pack()

root.mainloop() # Inițiază bucla principală de evenimente pentru aplicația Tkinter

```